

EXERCISE-2

Q1) Create a object calculator, which will have methods read, add, subtract and multiply. Read method will use prompt to ask two values from user.

Simple Calculator

Enter Numbers

Add

Subtratct

Multiply

Output

Multiplication= 1000

Simple Calculator

Enter Numbers

Add

Subtratct

Multiply

Output

Addition= 70

Simple Calculator

Enter Numbers

Add

Subtratct

Multiply

Output

Subtraction= 30

Q2) Create an object temperatureConverter with methods:

- **read()** → Asks the user to enter a temperature in Celsius.
- **toFahrenheit()** → Converts it to Fahrenheit.
- **toKelvin()** → Converts it to Kelvin.
- **display()** → Displays the results in the console

Temperature Converted

Enter Temperature

Fahrenheit

Kelvin

Display Result

Output

Temperatures:

Temperature In Celsius: 30°
Temperature In Fahrenheit: 86°
Temperature In Kelvin: 303.15°

Q3)Tasks:

- Predict the output of the given snippet.
- Explain how hoisting affects the execution of `console.log(z)` in `second()`.
- Explain the scope chain for `console.log(y)` in `second()`.
- What happens when `console.log(y)` is executed outside `first()` (Line 4)? Why?
- Modify the code to use `let` instead of `var` and observe any differences.

```
var x = 5;

function first() {
  console.log(x); // Line 1
  var y = 10;

  function second() {
    console.log(y); // Line 2
    console.log(z); // Line 3
    var z = 20;
  }

  second();
}

first();
console.log(y); // Line 4
```

Answers:

i)output of the given snippet.

Predicted Output(in order):

- 5

Explanation- 5 gets printed because in the global scope `var x=5` is declared and initialized, when `console.log(x)` couldn't find `x` in the function scope it went to search in the parent scope which in this case was the global scope. It shows that (`var`) is function scoped.

- 10
Explanation- 10 gets printed because it was declared and initialised in first(), when the code is in second() it tries to find (y) when it doesn't find y in second(), it goes to the parent scope which in this case is first(), inside that y is declared and declared therefore printing 10.
- Undefined
Explanation- Undefined gets printed because it was called in second() console.log(z) but var z=20 is declared and initialized in the next line. Even though due to hoisting var z gets declared on the first line inside second() but it does not get initialized there.
- ReferenceError
Explanation- Reference Error gets printed because var y was declared inside the function first(), it is function scoped and console.log(y) is trying to access it outside of the function, there is no global scope of y, i.e, it throws reference error.

ii)how hoisting affects the execution of console.log(z) in second().

- Var z is hoisted to the top of second().
- It gets declared but not initialized i.e(z=20 is not hoisted to the top only var z does)
- That is the reason console.log(z) prints undefined.

iii)Explain the scope chain for console.log(y) in second().

JavaScript looks for y in this order:

- Local scope of second() → not found
- Parent scope (first()) → found (var y = 10)
- Global scope → not needed

iv)What happens when `console.log(y)` is executed outside `first()` (Line 4)? Why?

- `y` was declared using `var` inside `first()`
- Function scope means `y` does not exist outside `first()`
- Global scope cannot access it

v)Modify the code to use `let` instead of `var` and observe any differences.

```
>> let x=5;

function first(){
  console.log(x);
  let y=10;

  function second(){
    console.log(y);
    console.log(z);
    let z=10;
  }
  second();
}
first();
console.log(y);
```

5

10

! ▶ Uncaught ReferenceError: can't access lexical declaration 'z' before initialization

second	debugger eval code:9
first	debugger eval code:12
<anonymous>	debugger eval code:14

Explanation- Changing to `let` from `var` changes `console.log(z)` from giving the output undefined to reference error because when declaring a variable using (`let`) , in the function `second()` `z` gets hoisted up to the first line inside `second()` it gets hoisted and due to using it is now inside a Temporal Dead Zone(TDZ-time between scope creation and variable initialization) .

Here it is trying to access `z` in `console.log(z)` before their declaration which results in `ReferenceError`.